

Sometimes You Need to be Aggressive: Improving Signal’s PQ Ratchet for Imbalanced Communications

Artifacts Appendix

1. Extended Proof of UKatana

For the sake of completeness, we include the full proof of UKatana, highlighting the main differences between the proof of Katana [1]

1.1. Proof of FS-IND-CPA Security

As mentioned in the proof of Thm. 5.1 in the main body, the proof follows a similar proof to the FS-IND-CPA security of Katana [1, Theorem 6.2].

Proof. Due to the symmetry of users A and B, we only focus on bounding the advantage $\text{Adv}_{\mathcal{A}}^{\text{FS-IND-CPA-A}}(1^\lambda, i^*)$ (cf. Def. 4.3). The theorem is proven in a sequence of hybrid games. The first Game₀ is the real FS-IND-CPA security game, where Game₆ is a game in which even an unbounded adversary has negligible advantage. Our proof consists of bounding the advantage of an adversary $\mathcal{A}$ of the adjacent games. Below, ϵ_i denotes the advantage of $\mathcal{A}$ in Game _{i} and Q denotes the number of random oracle queries performed by $\mathcal{A}$.

Game₀: This is the real FS-IND-CPA security game.

Game₁: In this game, the challenger reuses K, s, e from algorithm RUEnc-A as opposed to generating them through executing RUDec-B. This follows from the same argument made to prove correctness: m used during RUEnc-A and m' generated during RUDec-B are the same with all but a negligible probability. Hence, we have

$$|\epsilon_0 - \epsilon_1| \leq \epsilon_{\text{corr}},$$

where ϵ_{corr} is the probability that correctness with updated keys fails (cf. Def. 4.2).

Game₂: In this game, the challenger first samples $(K_{i^*,0}, \hat{s}_A, \hat{e}_A, \hat{s}_B, \hat{e}_B) \xleftarrow{\$} \{0,1\}^\lambda \times (\chi_0)^4$ and later programs the random oracle H on input seed. In case the input is already queried, the challenger declares the adversary $\mathcal{A}$ wins and outputs 1 as the output of the game. This game is identical to Game₁ as long as such seed* — the seed sampled by the challenger to construct the challenge ciphertext ct_{A,i^*} — is not queried on the time of the creation of ct_{A,i^*} . Since

seed* is sampled uniformly random over $\{0,1\}^\lambda$, the probability of this occurring is $Q/2^\lambda$. Hence, we have

$$|\epsilon_1 - \epsilon_2| \leq \frac{Q}{2^\lambda}.$$

Game₃: In this game, the challenger no longer programs the random oracle. Instead, it aborts the game and declares the adversary wins when the random oracle is queried on seed*. We denote this event by E_3 . Clearly, as long as event E_3 does not occur, Game₂ and Game₃ proceed identically. Hence, we have

$$|\epsilon_2 - \epsilon_3| \leq \Pr[E_3].$$

As we cannot bound $\Pr[E_3]$ yet, we postpone bounding it to later.

Game₄: In this game, the challenger computes user A’s updated key ek_{A,i^*}^1 directly, without using ek_{A,i^*-1}^1 . Namely, it directly computes the secret and noise:

$$\begin{aligned} \text{dk}_{A,i^*}^1 &:= s_A + \sum_{i \in [0:i^*-1]} \hat{s}_A^i \\ \text{e}_{A,i^*}^1 &:= e_A + \sum_{i \in [0:i^*-1]} \hat{e}_A^i, \end{aligned}$$

and sets $\text{ek}_{A,i^*}^1 = \mathbf{D} \cdot \text{dk}_{A,i^*}^1 + \text{e}_{A,i^*}^1$. Here, note that the initial secret and noise *before* the update s_A and e_A , respectively, sampled from $\mathcal{D}_{\text{RUKemGen-B}}(0)$ is unknown to the adversary. Since this is only a conceptual change, we have

$$\epsilon_3 = \epsilon_4.$$

Moreover, denoting E_4 the event that the adversary triggers the abort condition — defined in the previous game — in Game₄, we also have

$$\Pr[E_3] = \Pr[E_4].$$

Game₅: This is the new part of the proof unique to RUKEM. For RUKEM [1, Theorem 6.3], they were able to simply swap party B’s encapsulation key to random. However, we can no longer do this as explained in our proof in Thm. 5.1. We therefore, resort to *hint* MLWE.

More specifically, in this game, the challenger samples a random $\text{ek}_B^{i^*+1} := \mathbf{u}_B^{i^*+1}$ from $\mathcal{R}_q^k$ as opposed to generating them as

$$\begin{aligned} & \mathbf{u}^{i^*} + (\mathbf{D}^\top \cdot \hat{\mathbf{s}}_B + \hat{\mathbf{e}}_B) \\ &= \mathbf{D}^\top \cdot \left(\mathbf{s}_B + \sum_{i \in [0:i^*]} \hat{\mathbf{s}}_B^i \right) + \left(\mathbf{e}_B + \sum_{i \in [0:i^*]} \hat{\mathbf{e}}_B^i \right) \\ &= \underbrace{\text{ek}_B^0}_{=\mathbf{D}^\top \cdot \mathbf{s}_B + \mathbf{e}_B} + \mathbf{D}^\top \cdot \left(\sum_{i \in [0:i^*]} \hat{\mathbf{s}}_B^i \right) + \left(\sum_{i \in [0:i^*]} \hat{\mathbf{e}}_B^i \right), \end{aligned}$$

where the initial key *before* the update ($\text{ek}_B^0, \text{dk}_B^0 := \mathbf{s}_B$) $\stackrel{\$}{\leftarrow} \mathcal{D}_{\text{RUKKeyGen-B}}(0)$ is unknown to the adversary. Here, we use the fact that $\mathcal{D}_{\text{RUKKeyGen-B}}(1) = [2] \cdot \mathcal{D}_{\text{RUKKeyGen-B}}(0)$. Otherwise, this is the same game as before. In particular, the adversary is still provided with $\text{dk}_B^{i^*+1} = \mathbf{s}_B + \sum_{i \in [0:i^*]} \hat{\mathbf{s}}_B^i$.

It is easy to check that Game_5 is indistinguishable from Game_4 under the hint-MLWE assumption, where $\text{ek}_B^0 = \mathbf{D}^\top \cdot \mathbf{s}_B + \mathbf{e}_B$ is the MLWE instance and $\mathbf{h} = \mathbf{s}_B + \hat{\mathbf{s}}_B$ the *hint*. From the hint, the challenger can simulate $\text{dk}_B^{i^*+1} = \mathbf{h} + \sum_{i \in [0:i^*-1]} \hat{\mathbf{s}}_B^i$. Here, we highlight that the way we modified the behavior of the random oracle is crucial since the challenger does not need to know the output of $H(\text{seed}^*)$ regarding B's update ($\hat{\mathbf{s}}_B, \hat{\mathbf{e}}_B$) conditioning on the game not aborting.

Let E_5 denote the event that $\mathcal{A}$ triggers an abort in Game_5 and let Win_i denote the event that $\mathcal{A}$ wins in Game_i . Then, we can construct an adversary $\mathcal{B}_{\text{hint-MLWE},1}$ against the hint-MLWE $_{q,k,2k,\chi_0,\chi_0,\mathcal{F}_{\text{cpa}}}$ problem such that

$$|\Pr[\text{Win}_4 \wedge \neg E_4] - \Pr[\text{Win}_5 \wedge \neg E_5]| \leq \text{Adv}_{\mathcal{B}_{\text{hint-MLWE},1}}^{\text{hint-MLWE}}(1^\lambda).$$

Moreover, observe we have

$$\begin{aligned} & |\epsilon_4 - \epsilon_5| \\ &= |(\Pr[E_4] \cdot \Pr[\text{Win}_4|E_4] + \Pr[\text{Win}_4 \wedge \neg E_4]) \\ &\quad - (\Pr[E_5] \cdot \Pr[\text{Win}_5|E_5] + \Pr[\text{Win}_5 \wedge \neg E_5])| \\ &\leq \Pr[E_4] + \Pr[E_5] + |\Pr[\text{Win}_4 \wedge \neg E_4] - \Pr[\text{Win}_5 \wedge \neg E_5]| \\ &\leq \Pr[E_4] + \Pr[E_5] + \text{Adv}_{\mathcal{B}_{\text{hint-MLWE},1}}^{\text{hint-MLWE}}(1^\lambda), \end{aligned}$$

where we use the fact that $\Pr[\text{Win}_i|E_i] = 1$ for $i \in \{4, 5\}$. In addition, it can be checked that the differences between $\Pr[E_4]$ and $\Pr[E_5]$ are negligible assuming the hardness of the hint-MLWE problem. Formally, we can construct an adversary $\mathcal{B}_{\text{hint-MLWE},2}$ against the hint-MLWE $_{q,k,2k,\chi_0,\chi_0,\mathcal{F}_{\text{cpa}}}$ problem such that

$$|\Pr[E_4] - \Pr[E_5]| \leq \text{Adv}_{\mathcal{B}_{\text{hint-MLWE},2}}^{\text{hint-MLWE}}(1^\lambda).$$

As we cannot bound $\Pr[E_5]$ yet, this is postponed to the following final game.

Game₆: In the final game, the challenger samples a random $\text{ek}_{A,i^*-1}^1 := \mathbf{u}_A$ from $\mathcal{R}_q^k$. Moreover, it samples random v'_B from $\mathcal{R}_q$ and sets the ciphertext as $\text{ct}_B := v'_B + m$. This part of the proof is the same

as the final step in Katana [1, Theorem 6.3] as it only concerns A's key update. Therefore, relying on the same proof, we can construct an adversary $\mathcal{B}'_{\text{hint-MLWE},1}$ against the hint-MLWE $_{q,k,2k,\chi_0,\chi_0,\mathcal{F}_{\text{cpa}}}$ problem with $\mathcal{F} := \mathcal{U}(\{\mathbf{I}_{2k \times 2k}\})$ such that

$$|\epsilon_5 - \epsilon_6| \leq \Pr[E_5] + \Pr[E_6] + \text{Adv}_{\mathcal{B}'_{\text{hint-MLWE},1}}^{\text{hint-MLWE}}(1^\lambda).$$

In addition, we can construct an adversary $\mathcal{B}'_{\text{hint-MLWE},2}$ against the hint-MLWE $_{q,k,2k,\chi_0,\chi_0,\mathcal{F}_{\text{cpa}}}$ problem such that

$$|\Pr[E_5] - \Pr[E_6]| \leq \text{Adv}_{\mathcal{B}'_{\text{hint-MLWE},2}}^{\text{hint-MLWE}}(1^\lambda).$$

Lastly, observe that in the final game, m is information theoretically hidden from $\mathcal{A}$ as v'_B is sampled uniformly at random. Put differently, seed^* is hidden from $\mathcal{A}$, and thus, we have $\Pr[E_6] \leq Q/2^\lambda$. Combining everything together, we establish

$$|\epsilon_5 - \epsilon_6| = \text{negl}(\lambda).$$

□

1.2. Proof of Ratchet Simulatability

As mentioned in the proof of Thm. 5.2 in the main body (cf. Secs. B and 5.2.2), the proof follows a similar proof to the ratchet simulability of Katana [1, Theorem 6.3].

Proof. The simulators $(\text{RUSimKey-P}_1, \text{RUSimKey-P}_2, \text{RUSimKey-P}_3, \text{RUSimCtxt-P})_{P \in \{A,B\}}$ used to prove ratchet simulability is provided in Fig. 14 of the main body. As we already established baee key, sender and receiver key simulatability, we focus on ciphertext simulatability.

Below, due to the symmetry of A and B, we only focus on the distinguishing advantage of the distributions $\mathcal{D}_{B,0}^{\text{CtxtSim}}$ and $\mathcal{D}_{B,1}^{\text{CtxtSim}}$. This is proven in a sequence of hybrid distributions. Let ϵ_i denote the probability that $\mathcal{A}$ outputs 1 given a sample from $\mathcal{D}_{B,i}$. The goal is then to bound the difference between ϵ_0 and ϵ_6 .

Distribution $\mathcal{D}_{B,0}$: This is the distribution $\mathcal{D}_{B,0}^{\text{CtxtSim}}$.

Distribution $\mathcal{D}_{B,1}$: The only difference between the prior distribution is that we reuse $(K, \hat{\mathbf{s}}_A, \hat{\mathbf{e}}_A, \hat{\mathbf{s}}_B, \hat{\mathbf{e}}_B)$ from algorithm RUEnc-B as opposed to generating them through executing RUDec-A . This follows from correctness: m used during RUEnc-B and m' generated during RUDec-A are the same with all but a negligible probability. Namely, we have

$$|\epsilon_0 - \epsilon_1| \leq \epsilon_{\text{corr}},$$

where ϵ_{corr} is the probability that correctness with updated keys fails (cf. Def. 4.2).

Distribution $\mathcal{D}_{B,2}$: The main difference between the prior distribution is that we compute the ciphertext $\text{ct}_B = v_B$ in a different way. Below, we show that these two ways

of computing v_B are identical, where the first equality is how v_B is computed in the prior distribution:

$$\begin{aligned} v_B - m &= \text{ek}_A^{i^* \top} \cdot \mathbf{s}_B + \tilde{e}_B \\ &= (\mathbf{D} \cdot \mathbf{s}_A^{i^*} + \mathbf{e}_A^{i^*})^\top \cdot \mathbf{s}_B + \tilde{e}_B \\ &= \mathbf{s}_A^{i^* \top} \cdot (\mathbf{D}^\top \cdot \mathbf{s}_B + \mathbf{e}_B) - \mathbf{s}_A^{i^* \top} \cdot \mathbf{e}_B + \mathbf{e}_A^{i^* \top} \cdot \mathbf{s}_B + \tilde{e}_B \\ &= \mathbf{s}_A^{i^* \top} \cdot \text{ek}_B^0 - \mathbf{s}_A^{i^* \top} \cdot \mathbf{e}_B + \mathbf{e}_A^{i^* \top} \cdot \mathbf{s}_B + \tilde{e}_B, \end{aligned}$$

where the secret and noise $\mathbf{s}_B$ and $\mathbf{e}_B$ corresponds to those sampled from $\mathcal{D}_{\text{RUKemGen-B}}(0)$. Concretely, the last equation is exactly how v_B is computed in $\mathcal{D}_{B,2}$. As this is merely a conceptual change, we have

$$\epsilon_1 = \epsilon_2.$$

Distribution $\mathcal{D}_{B,3}$: The difference between the prior distribution is that we sample $(K, \hat{\mathbf{s}}_A, \hat{\mathbf{e}}_A, \hat{\mathbf{s}}_B, \hat{\mathbf{e}}_B) \xleftarrow{\$} \{0, 1\}^\lambda \times (\chi_0)^4$ and later program the random oracle H on input seed. In case the input is already queried, the distribution outputs a special symbol $\top$. This distribution is identical to the previous one as long as $\mathcal{A}$ does not query the random oracle on seed. Since seed is sampled uniformly random over $\{0, 1\}^\lambda$, the probability of this occurring is $Q/2^\lambda$.

Hence, we have

$$|\epsilon_2 - \epsilon_3| \leq \frac{Q}{2^\lambda}.$$

Distribution $\mathcal{D}_{B,4}$: The difference between the prior distribution is that we sample a random $\mathbf{u}_B$ from $\mathcal{R}_q^k$ and sets user B's key as $\text{ek}_B := \mathbf{u}_B$. Recall in the previous game, $\mathbf{u}_B$ was set as MLWE instances. Since $\mathbf{s}_B$ and $\mathbf{e}_B$ are partially leaked via $\text{ct}_A = v_A$ and $\text{dk}_A^{i^*} = \mathbf{s}_A^{i^*}$, we cannot rely on the standard MLWE assumption to argue indistinguishability of the two distributions. However, noticing that $\tilde{e} \xleftarrow{\$} \tilde{\chi}$ is information theoretically hidden to $\mathcal{A}$ conditioning on the game not aborting, we can rely instead on the *hint* MLWE assumption. Concretely, we can construct an adversary $\mathcal{B}_{\text{hint-MLWE}}$ against the $\text{hint-MLWE}_{q,k,1,\chi_0,\tilde{\chi},\mathcal{F}_{\text{sim}}}$ problem such that

$$|\epsilon_3 - \epsilon_4| \leq \text{Adv}_{\mathcal{B}_{\text{hint-MLWE}}}^{\text{hint-MLWE}}(1^\lambda),$$

where recall a sample from $\mathcal{F}_{\text{sim}}$ has the form $[-\hat{\mathbf{s}}^\top | \hat{\mathbf{e}}^\top]$ with $\hat{\mathbf{s}}, \hat{\mathbf{e}} \xleftarrow{\$} \chi_{i^*} := [i^* + 1] \cdot \chi_0$. In more detail, $\mathcal{B}_{\text{hint-MLWE}}$ obtains

$$(\mathbf{D}^\top, \mathbf{u}_B, [-\hat{\mathbf{s}}^\top | \hat{\mathbf{e}}^\top], h = [-\hat{\mathbf{s}}^\top | \hat{\mathbf{e}}^\top] \begin{bmatrix} \mathbf{e}_B \\ \mathbf{s}_B \end{bmatrix} + \tilde{e}_B)$$

as input, where $\mathbf{u}_B$ is either random or of the form $\mathbf{D}^\top \cdot \mathbf{s}_B + \mathbf{e}_B$. Here h is the *hint*. Without loss of generality in the hardness of the hint-MLWE problem, we can assume we obtain $(\mathbf{s}_i, \mathbf{e}_i)_{i \in [0:i^*]}$, each tuple sampled from $\chi_0 \times \chi_0$ such that they sum to $(\hat{\mathbf{s}}, \hat{\mathbf{e}})$.

It then simulates each $(\text{ek}_A^i, \text{dk}_A^i)_{i \in [0:i^*]}$ using $(\mathbf{s}_i, \mathbf{e}_i)_{i \in [0:i^*]}$ and computes the ciphertext $\text{ct}_A = v_A = \hat{\mathbf{s}}^\top \cdot \mathbf{u}_B + h + m$. We highlight that this is the main game that differed from Katana [1,

Theorem 6.3] as we have to simulate all the keys for A and the noise incurred in the hints are larger.

Distribution $\mathcal{D}_{B,5}$: The only difference from the prior distribution is that we sample user B's first updated key (performed by himself) $\text{ek}_B^1 = \mathbf{u}_B^1$ uniformly sample and then set ek_B^0 . In the previous distribution, this was performed in the opposite direction. Since this produces an identical distribution, we have $\epsilon_4 = \epsilon_5$.

Distribution $\mathcal{D}_{B,6}$: The only difference from the prior distribution is that we sample user B's updated key $\text{ek}_B^1 = \mathbf{u}_B^1$ as a valid MLWE instance. It is straight forward to see that the two distributions is indistinguishable under the MLWE assumption. Formally, we can construct an adversary $\mathcal{B}_{\text{MLWE}}$ against the MLWE_{q,k,χ_1} problem such that

$$|\epsilon_5 - \epsilon_6| \leq \text{Adv}_{\mathcal{B}_{\text{MLWE}}}^{\text{MLWE}}(1^\lambda).$$

Lastly, it can be checked that the final distribution $\mathcal{D}_{B,6}$ is identical to $\mathcal{D}_{B,1}^{\text{CtxtSim}}$ as the generation of the updated key $(\text{ek}_B^1, \text{dk}_B^1)$ can be pushed before generating the non-updated key $(\text{ek}_B^0, \text{dk}_B^0)$. $\square$

2. SCKA Descriptions

We provide formal descriptions of our new SCKA protocols with aggressive sending strategies.

2.1. Agg-RUKEM

Agg-RUKEM's pseudocode is presented in Fig. 1. In Agg-RUKEM, party A's state contains the following variables (party B's state is symmetric):

old-dk _A	previous decapsulation key of A
old-ek _B	previous encapsulation key of B
new-dk _A	newest decapsulation key of A
new-ek _A	newest encapsulation key of A
new-ek _B	newest encapsulation key of B
upd-ek _B	update counter for old-ek _B
ct _A	ciphertext generated by A
chunks	list of received chunks
i _{snd}	counter for chunks sent by A
ack-ek _A	acknowledgment for new-ek _A
ack-ek _B	acknowledgment for new-ek _B
send-ct _A	flag A is permitted to send ct
send-ct _B	flag B is permitted to send ct
t	epoch
t _A	A's newest keypair index
t _B	B's newest keypair index

Note i_{ek} denotes the number of chunks necessary to transmit an encapsulation key.

2.2. Agg-UniKEM

A full description of the protocol logic for Agg-UniKEM can be found in the artifacts repository at `cpp_simulations/docs/agg_unikem_parallel_role_model.md`.

SCKA-Init-KeyGen(1^λ) <pre> 1: $\text{par} \xleftarrow{\\$} \text{RUSetup}(1^\lambda, k)$ 2: $(\text{ek}_A^{-1}, \text{dk}_A^{-1}) \xleftarrow{\\$} \text{RUKeyGen-A}(\text{par}, \text{updated})$ 3: $(\text{ek}_B^{-1}, \text{dk}_B^{-1}) \xleftarrow{\\$} \text{RUKeyGen-B}(\text{par}, \text{updated})$ 4: return $\text{I}_{\text{SCKA}} := (\text{par}, \text{ek}_A^{-1}, \text{dk}_A^{-1}, \text{ek}_B^{-1}, \text{dk}_B^{-1})$ </pre>	SCKA-Rec-B(ρ) <pre> 1: $I_B \leftarrow \perp, t_{I_B} \leftarrow \perp$ 2: <i>/ If A received B's ciphertext and expects a new one</i> 3: if $\rho.t_B > t_B$ 4: <i>/ Advance sending index</i> 5: $(\text{ack-ek}_B, \text{send-ct}_B) \leftarrow (\text{false}, \text{false})$ 6: $\text{ct}_B \leftarrow \perp$ 7: $\text{new-ek}_B \leftarrow \perp$ 8: $t_B \leftarrow 1, t \leftarrow 1$ 9: $i_{\text{snd}} \leftarrow 0$ 10: <i>/ Process received chunk</i> 11: if $\llbracket \rho.t_A = t_A \rrbracket$ 12: if $\llbracket \neg \text{ack-ek}_A \rrbracket$ 13: $\text{chunks} \xleftarrow{\perp} \{\rho.\text{ch}\}$ 14: <i>/ Decode A's public key</i> 15: if $\llbracket \text{Decode}(\text{chunks}) \neq \perp \rrbracket$ 16: $\text{new-ek}_A \leftarrow \text{Decode}(\text{chunks})$ 17: $\text{ack-ek}_A \leftarrow \text{true}$ 18: $\text{chunks} \leftarrow \emptyset$ 19: elseif $\llbracket \rho.b = 1 \wedge \rho.\text{ch} \neq \perp \rrbracket$ 20: $\text{chunks} \xleftarrow{\perp} \{\rho.\text{ch}\}$ 21: if $\llbracket \text{Decode}(\text{chunks}) \neq \perp \rrbracket$ 22: $\text{ct}_A \leftarrow \text{Decode}(\text{chunks})$ 23: $(\text{old-dk}_B, \text{old-ek}_A, I_A) \leftarrow \text{RUDec-B}(\text{old-dk}_B, \text{new-ek}_A, \text{ct}_A)$ 24: $\text{upd-ek}_A \leftarrow 1$ 25: $t_{I_A} \leftarrow t$ 26: $\text{new-ek}_A \leftarrow \perp$ 27: $(\text{ack-ek}_A, \text{send-ct}_A) \leftarrow (\text{false}, \text{false})$ 28: $t_A \leftarrow 1, t \leftarrow 1$ 29: $\text{chunks} \leftarrow \emptyset$ 30: <i>/ Check if B's ek/dk was acknowledged by A</i> 31: if $\llbracket \rho.t_B = t_B \rrbracket$ 32: if $\llbracket \rho.\text{ack-ek}_B \rrbracket$ 33: $\text{ack-ek}_B \leftarrow \text{true}$ 34: if $\llbracket \rho.\text{send-ct}_B \rrbracket$ 35: $\text{send-ct}_B \leftarrow \text{true}$ 36: return $((t_{I_A}, I_A), t - 1)$ </pre>
SCKA-Init-A(I_{SCKA}) <pre> 1: $\text{parse}(\text{par}, \text{ek}_A, \text{dk}_A, \text{ek}_B, \text{dk}_B) \leftarrow \text{I}_{\text{SCKA}}$ 2: $(\text{old-dk}_A, \text{old-ek}_B) \leftarrow (\text{dk}_A, \text{ek}_B)$ 3: $(\text{new-dk}_A, \text{new-ek}_A, \text{new-ek}_B) \leftarrow (\perp, \perp, \perp)$ 4: $\text{upd-ek}_B \leftarrow 1$ 5: $\text{ct}_A \leftarrow \perp$ 6: $\text{chunks} \leftarrow \emptyset$ 7: $i_{\text{snd}} \leftarrow 0$ 8: $(\text{ack-ek}_A, \text{ack-ek}_B) \leftarrow (\text{false}, \text{false})$ 9: $(\text{send-ct}_A, \text{send-ct}_B) \leftarrow (\text{false}, \text{false})$ 10: $(t, t_A, t_B) \leftarrow (1, 1, 1)$ 11: $\text{st}_A \leftarrow (\text{old-dk}_A, \text{old-ek}_B, \text{new-dk}_A, \text{new-ek}_A, \text{new-ek}_B,$ $\text{upd-ek}_B, \text{ct}_A, \text{chunks}, i_{\text{snd}}, \text{ack-ek}_A,$ $\text{ack-ek}_B, \text{send-ct}_A, \text{send-ct}_B, t, t_A, t_B)$ 12: return st_A </pre>	
SCKA-Send-A(par) <pre> 1: $I_A \leftarrow \perp, t_{I_A} \leftarrow \perp, \text{ch} \leftarrow \perp, b \leftarrow 0$ 2: <i>/ Generate a new key pair if necessary</i> 3: if $\llbracket \neg \text{ack-ek}_A \wedge i_{\text{snd}} = 0 \wedge \text{upd-ek}_B \leq k + 1 \rrbracket$ 4: $(\text{new-ek}_A, \text{new-dk}_A) \xleftarrow{\\$} \text{RUKeyGen-A}(\text{par})$ 5: <i>/ Generate a chunk to send</i> 6: if $\llbracket \text{new-ek}_A \neq \perp \rrbracket$ 7: if $\llbracket \neg \text{ack-ek}_A \rrbracket$ 8: $b \leftarrow 0, i_{\text{snd}} \leftarrow 1$ 9: $\text{ch} \leftarrow \text{Encode}(\text{new-ek}_A, i)$ 10: elseif $\llbracket \text{send-ct}_A \rrbracket$ 11: if $\llbracket \text{ct}_A = \perp \rrbracket$ 12: $i_{\text{snd}} \leftarrow 0$ 13: $(\text{old-dk}_A, \text{old-ek}_B, \text{ct}_A, I_A) \leftarrow \text{RUEnc-A}(\text{new-dk}_A, \text{old-ek}_B)$ 14: $\text{upd-ek}_B \leftarrow 1$ 15: $\text{new-dk}_A \leftarrow \perp$ 16: $t_{I_A} \leftarrow t$ 17: $b \leftarrow 1, i_{\text{snd}} \leftarrow 1$ 18: $\text{ch} \leftarrow \text{Encode}(\text{ct}_A, i)$ 19: <i>/ Allow B to send their ciphertext</i> 20: if $\llbracket \text{ack-ek}_B \wedge \neg \text{send-ct}_A \wedge (i_{\text{snd}} < i_{\text{ek}} \vee t \text{ even}) \rrbracket$ 21: $\text{send-ct}_B \leftarrow \text{true}$ 22: $\rho \leftarrow (t, t_B, t_A, \text{ch}, \text{ack-ek}_B, b, \text{send-ct}_B)$ 23: return $((t_{I_A}, I_A), \rho, t - 1)$ </pre>	

Figure 1: Sparse continuous key-agreement Agg-RUKEM- k from k -RUKEM. Note that SCKA-Send-A and SCKA-Rec-B are implicitly stateful in st_A and st_B , respectively. SCKA-Init-B, SCKA-Send-B, and SCKA-Rec-A are defined analogously, with party names and the order of arguments when required, except for SCKA-Send-B using $\llbracket \dots \wedge (\dots \vee t \text{ odd}) \rrbracket$ on line 20.

References

- [1] Y. Dodis, D. Jost, S. Katsumata, T. Prest, and R. Schmidt, “Triple ratchet: A bandwidth efficient hybrid-secure signal protocol,” in *EUROCRYPT 2025, Part VIII*, ser. LNCS, S. Fehr and P.-A. Fouque, Eds., vol. 15608. Springer, Cham, May 2025, pp. 302–331.